LIVE MEETING SUMMARIZER APPLICATION

Module 1 Submission Report

---

1. Project Overview

The Live Meeting Summarizer Application is a Real-Time Speech-to-Text (STT) system designed to transcribe meeting audio into text and evaluate transcription performance.

The core objectives of this project are:

- **Implement real-time speech-to-text transcription**
- **Compare Whisper (Medium) and Vosk models**
- **Evaluate accuracy using Word Error Rate (WER)**
- **Benchmark CPU vs GPU performance**
- **Analyze latency and contextual retention**

This project demonstrates both real-time inference capability and systematic evaluation of speech recognition models.

---

2. Dataset Used

2.1 AMI Corpus

The dataset used for evaluation is from the AMI Meeting Corpus, a widely recognized dataset for meeting transcription research.

Audio File Used:

amicorpus – ES2002a.Array1-01.wav

This recording contains real-world meeting speech with:

- **Multiple speakers**
- **Natural conversational overlap**
- **Background noise**
- **Real meeting environment characteristics**

The dataset is suitable for evaluating both contextual understanding and robustness of STT models.

---

3. Audio Preprocessing

3.1 Audio Trimming

The original AMI audio file is long in duration.
For controlled evaluation, the audio was trimmed to 3 minutes.

Reason for Trimming:

- **Maintain consistent benchmarking**
- **Reduce computational cost**
- **Enable fair WER comparison**
- **Simulate real meeting segment**

Trimming Procedure:

1. **Load original .wav file**
2. **Extract first 180 seconds (3 minutes)**
3. **Save as processed evaluation sample**

This ensures standardized evaluation input for both models.

---

4. Transcript and Ground Truth

Ground truth transcript was obtained from the official AMI annotations.

Steps:

1. **Extract transcript corresponding to ES2002a.Array1-01.wav**
2. **Match transcript time range with trimmed 3-minute audio**
3. **Clean transcript (lowercase conversion, punctuation removal)**
4. **Normalize spacing**

This cleaned transcript is used as Ground Truth for WER calculation.

---

5. System Workflow

The project consists of the following workflow:

Step 1: Audio Input

- **Load trimmed audio file**
- **Perform preprocessing (normalization, noise handling)**

Step 2: Model Inference

- **Run transcription using Whisper (Medium)**
- **Run transcription using Vosk**

Step 3: Text Cleaning

- **Convert text to lowercase**
- **Remove punctuation**
- **Normalize whitespace**
- **Standardize format**

Step 4: Word Error Rate (WER) Calculation

- **Compare predicted transcript with ground truth**
- **Calculate substitutions, deletions, insertions**
- **Compute final WER value**

Step 5: Performance Benchmarking

- **Measure processing time**
- **Compare CPU vs GPU performance**

---

6. Models Used

6.1 Whisper (Medium)

- **Architecture: Transformer-based Encoder-Decoder**
- **Designed for contextual understanding**
- **Optimized for CUDA-enabled GPU**

- **Higher computational requirement**

Strength:

- **Excellent contextual retention**
- **Strong performance in noisy conditions**

6.2 Vosk

- **Architecture: Kalman Filter + Neural Network (WFST)**
- **Lightweight, offline speech recognition system**
- **Optimized for CPU usage**

Strength:

- **Extremely low latency**
- **Suitable for embedded systems**

Limitation:

- **Less contextual awareness**
- **Performance degrades in noisy multi-speaker scenarios**

---

7. Performance Comparison

Feature	Whisper (Medium)	Vosk
Architecture	Transformer-based (Encoder-Decoder)	Kalman Filter + Neural Network (WFST)
Word Error Rate (WER)	0.26 (after preprocessing)	Higher in noisy meeting audio
Hardware	CUDA / GPU optimized	CPU optimized
Transcription Quality	Excellent context retention	Fast but limited contextual depth
Processing Time	Slower (requires GPU for real-time)	Extremely fast (low latency)

---

8. Word Error Rate (WER) Evaluation

WER Formula:

WER = (Substitutions + Deletions + Insertions) / Total Words in Ground Truth

Procedure:

1. **Align predicted transcript with reference transcript**
2. **Count:**
    - **Substitutions**
    - **Insertions**
    - **Deletions**
3. **Compute WER**

Result:

Whisper achieved WER = 0.26 after preprocessing.

Vosk showed comparatively higher WER in noisy segments.

---

9. Real-Time Speech-to-Text Implementation

The system supports:

- **Microphone input**
- **Streaming audio transcription**
- **Live speech recognition**
- **Model-based prediction**

Real-time inference was tested for both models with attention to latency.

---

10. CPU vs GPU Benchmark

Observations:

- **Whisper requires GPU for efficient real-time performance.**

- **On CPU, Whisper processing time increases significantly.**
- **Vosk performs efficiently on CPU with minimal latency.**
- **Whisper GPU: 2.4 sec**
  Whisper CPU: 18.7 sec
  Vosk CPU: 1.3 sec

Conclusion:

For high-accuracy applications → Whisper + GPU
For low-resource real-time systems → Vosk + CPU

---

11. Complete Step-by-Step Implementation Summary

1. **Obtain AMI audio file**
2. **Trim audio to 3 minutes**
3. **Extract corresponding transcript**
4. **Clean transcript-removed punctuations,removed spaces and lowercase the alphabets**
5. **Run transcription using Whisper**
6. **Run transcription using Vosk**
7. **Normalize predicted text**
8. **Compute WER**
9. **Measure processing time**
10. **Compare results**
11. **Generate evaluation report**

---

12. Conclusion

This project successfully implements and evaluates a Real-Time Speech-to-Text system using two distinct architectures.

Key Takeaways:

- **Transformer-based models provide superior contextual accuracy.**

- **Lightweight WFST-based models offer faster processing with lower hardware demands.**
- **WER is an effective metric for transcription evaluation.**
- **Hardware plays a critical role in real-time STT performance.**

This project fulfills the Module 1 objectives:

- **Real-Time STT implementation**
- **Model comparison**
- **WER evaluation**
- **CPU vs GPU benchmarking**
- **Clean methodological documentation**

SCREENSHOT OF WER(ON KAGGLE):